

# Atelier — Pentest live sur eRezo

## Architecture IA · Année 5 · Séance pratique (2h30)

---

### La mission

Par **binôme (2 personnes maximum)**, vous menez un **test d'intrusion en conditions réelles** sur l'application <https://erezo.fr>.  
Objectif : jouer le rôle d'un attaquant authentifié et **trouver un maximum de failles de sécurité**, les prouver, et les prioriser — comme le ferait un pentesteur professionnel.

Vous ne cassez rien : vous **cherchez, testez en lecture seule, et documentez**.

---

### ⚠ Cadre légal — À LIRE AVANT DE COMMENCER

*Un test d'intrusion sur un système que vous ne possédez pas est un DÉLIT. En France, l'accès ou le maintien frauduleux dans un système informatique est puni par les articles 323-1 et suivants du Code pénal (jusqu'à plusieurs années de prison et de lourdes amendes). L'intention ne protège pas : tester sans autorisation suffit à être en infraction.*

**Pourquoi cet exercice est-il légal ?** Parce que **eRezo.fr appartient à KodeSaaS**, qui **autorise explicitement** ce test dans un cadre pédagogique et encadré. Cette autorisation est la SEULE chose qui rend l'exercice licite.

**Ce que vous ne ferez JAMAIS en dehors de ce cours :** reproduire ces techniques sur un site tiers (celui d'une entreprise, d'un concurrent, d'un ami...) sans une autorisation écrite. Pas d'exception.

---

### Étape 1 — Créez votre compte de test

Chaque binôme crée **son propre compte** sur <https://erezo.fr> (inscription classique). Ce compte est votre point de départ : vous testez en tant qu'utilisateur authentifié « lambda ».

Notez vos identifiants de test ici : \_\_\_\_\_

---

### Les règles du jeu (impératives)

#### ✅ Autorisé

- Reconnaissance passive : en-têtes HTTP, `robots.txt`, `sitemap.xml`, code source de la page (DevTools).
- Navigation authentifiée avec **votre compte de test**.
- Manipulation d'URL et de paramètres (changer un identifiant, un champ) **en lecture seule**.
- Analyse des en-têtes de sécurité, cookies, formulaires, appels réseau (onglet Réseau des DevTools).

#### 🚫 Interdit

- Toute action **destructive** : suppression, modification massive, corruption de données.
- Brute-force** de mots de passe, **déni de service (DoS)**, flood de requêtes.
- Exfiltrer ou stocker de vraies données** d'autres utilisateurs. Si vous croisez des données réelles → **anonymisez** (« Utilisateur A »).
- Sortir du périmètre : **uniquement erezo.fr**, rien d'autre.

En cas de comportement inattendu (erreur serveur, donnée modifiée par erreur) : **arrêtez et prévenez le formateur**. Signaler ne pénalise pas.

---

### Méthodologie — les 5 phases d'un pentest

Suivez cet ordre. À chaque phase, cherchez, et **notez tout ce qui vous semble anormal** dans la grille plus bas. (On ne vous dit pas *quelles* failles existent : c'est à vous de les débusquer.)

## Phase 1 — Reconnaissance

Que révèle l'application sans même se connecter ? Regardez les **en-têtes HTTP** de la réponse, le `robots.txt`, le `sitemap.xml`, les technologies utilisées (indices dans les en-têtes, le code source). *Question à se poser : le serveur en dit-il trop sur lui-même ? Y a-t-il des chemins « cachés » listés quelque part ?*

## Phase 2 — Cartographie authentifiée

Connectez-vous avec votre compte de test. Listez les **pages et fonctionnalités** : profil, messagerie, contacts, événements, images/avatars... Ouvrez l'onglet **Réseau** des DevTools et observez les **appels d'API** déclenchés. *Question : quelles URL sont appelées ? Que renvoient-elles ?*

## Phase 3 — Contrôle d'accès (le plus important)

C'est la faille n°1 des applications web. Repérez les URL qui contiennent un **identifiant** (un numéro, un nom de fichier). Que se passe-t-il si vous **changez cet identifiant** par un autre ? *Question : accédez-vous à des données qui ne sont pas les vôtres ? Une ressource est-elle accessible sans être connecté ?*

## Phase 4 — En-têtes & configuration

Analysez les **en-têtes de sécurité** de l'application (il existe des outils en ligne pour les scanner, ou l'onglet Réseau). *Question : les en-têtes de protection sont-ils présents et correctement configurés ? Le serveur expose-t-il des informations sensibles ?*

## Phase 5 — Injection & entrées utilisateur

Testez les **champs de saisie** (formulaires de contact, de profil...). Que deviennent vos entrées quand elles sont réaffichées ? *Question : une entrée « piégée » est-elle correctement neutralisée partout où elle réapparaît ?*

 **Vous pouvez vous faire assister par l'IA** (méthodo, interprétation d'un en-tête, idée de test) — mais **vérifiez toujours** : une faille « hallucinée » non reproduite ne compte pas.

## Pour vous repérer — l'OWASP Top 10 (en français)

L'**OWASP Top 10** est la liste de référence des 10 familles de failles web les plus critiques. Gardez-la sous les yeux pendant votre pentest :

- Cloudflare (FR) — <https://www.cloudflare.com/learning/security/threats/owasp-top-10/> (le plus pédagogique)
- OWASP officiel (2021) — <https://owasp.org/Top10/2021/> (la source de référence)
- Présentation gouv.fr — <https://pad.numerique.gouv.fr/s/MiBQqv42x>

## Votre grille de findings (à remplir)

Pour chaque faille trouvée, remplissez une ligne. **Une preuve = une faille qui compte.**

| # | Faillle (titre court) | Où (URL / zone) | Preuve (ce que vous avez fait/observé) | Impact | Priorité (P0/P1/P2) |
|---|-----------------------|-----------------|----------------------------------------|--------|---------------------|
| 1 |                       |                 |                                        |        |                     |
| 2 |                       |                 |                                        |        |                     |
| 3 |                       |                 |                                        |        |                     |
| 4 |                       |                 |                                        |        |                     |
| 5 |                       |                 |                                        |        |                     |

**Priorité : P0** = critique (exploitable à distance, gros impact, ex. fuite de données) · **P1** = important · **P2** = à durcir.

---

## Restitution (débrief)

En fin de séance, chaque binôme présente **ses 2-3 meilleures failles** : la preuve, l'impact, et la priorité. On compare, puis le formateur révèle le **corrigé complet**. Objectif : voir qui a trouvé les failles **critiques (P0)** — celles qui, combinées, exposent les données personnelles de tous les membres.

---

## 💰 Pour aller plus loin — le bug bounty

Vous voulez pentester **légalement** — et être **payé** pour ça ? C'est le **bug bounty** (« prime aux bugs »). Une entreprise publie un **périmètre autorisé** (scope) et des règles ; vous y cherchez des failles, et chaque faille valide est **récompensée** (souvent en argent). Variante sans prime : le **VDP** (*Vulnerability Disclosure Program*) — même cadre d'autorisation, simple signalement responsable.

C'est exactement le principe de cet atelier, mais à **l'échelle du monde réel** : ici, l'autorisation est **publique et cadrée** — vous ne sortez jamais du scope.

### Où trouver des cibles autorisées :

- **YesWeHack** — <https://yeswehack.com/programs> (plateforme française/européenne)
- **FireBounty** — <https://firebounty.com/> (agrégateur de tous les programmes + leur scope)
- **Bugcrowd** — <https://www.bugcrowd.com/bug-bounty-list/>
- **disclose.io** — <https://disclose.io/> (les politiques d'autorisation / safe harbor)

⚠️ Même en bug bounty, la règle d'or ne change pas : **on ne teste que ce qui est dans le périmètre autorisé du programme**. Tout le reste reste un délit.

---

Bon pentest — et **anonymisez tout** ce que vous croisez. 🔍